

# Introduction and Classical Cryptography

## CS 171: Cryptography

Sanjam Garg

August 27, 2026

# Overview

- 1 Course Information
- 2 From Art to Science
- 3 Private-Key Encryption
- 4 Classical Ciphers and Their Weaknesses
- 5 Summary

- **Course website:** <https://sanjamg.github.io/cs171-fall126/> (schedule, readings, policies, and links to everything below).
- **Lectures:** Tue/Thu, 12:30–2:00pm, Moffitt Library 145.
- **Ed:** all course announcements and Q&A — please join!
- **Gradescope:** homework and exams are submitted here (Course ID 1362375, entry code VEG54G).

- **Instructor:** Sanjam Garg (sanjamg@berkeley.edu).
- **Teaching Assistants:**
  - Diogo Antunes (dsantunes@berkeley.edu)
  - Zoë Ruha Bell (zbell@berkeley.edu)
- **DSP accommodations:** please have your letter sent to the instructor within the first two weeks, and contact the course staff about homework/exam logistics.

# Prerequisites

- The only formal prerequisite is discrete mathematics (**CS 70**).
- We assume familiarity with basic (discrete) **probability** and **modular arithmetic**.
- Some exposure to algorithms: comfortable reading pseudocode and familiar with big- $O$  notation.

# Homework

- Homeworks (and quizzes) are submitted on Gradescope, due at **8:59pm** on the due date.
- **Late policy:** lose 5% for every hour late — no exceptions.  
(1 minute late  $\Rightarrow$  -5%; 61 minutes late  $\Rightarrow$  -10%.)
- **Drop policy:** best  $x - 3$  out of  $x$  homeworks (e.g. best 7 out of 10).
- **Collaboration:** discussing problems with classmates is encouraged, but you must write up solutions *on your own*, never possess anyone else's solutions, and acknowledge all collaborators and sources.

Full policies (late submissions, collaboration, honor code) are on the course website.

# Exam Dates and Policies

- **Midterm I:** Thu Sep 24 (during class, Moffitt Library 145).
- **Midterm II:** Thu Oct 29 (during class, Moffitt Library 145).
- **Final:** Fri Dec 18, 8am–11am (location TBD).
- **Oral exams:** one week before each midterm, and during the last week of classes (before the final).
- All exams are **mandatory** — please keep these time slots free.

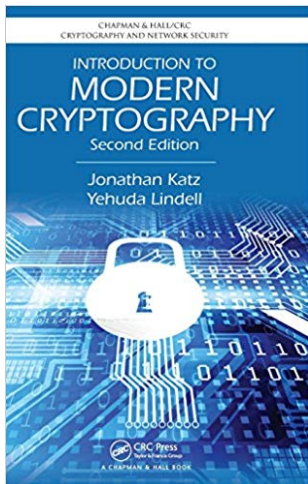
# Grading

- Homeworks and quizzes — 10%
- Midterm I — 25%
- Midterm II — 25%
- Final — 40%

## Clobber policy (lowest midterm)

Your lowest midterm score is replaced by the max of that score and your final-exam z-score mapped onto that midterm. Your other midterm is unchanged.

Homework scores could potentially be normalized based on the oral exams.



- Katz and Lindell, *Introduction to Modern Cryptography* ("KL"). Any edition is fine.
- Readings for each lecture are listed on the course website.  
Today: KL 1.1–1.3.
- Other excellent (mostly free) resources are linked from the website, e.g. Boneh–Shoup, *A Graduate Course in Applied Cryptography*, and Rosulek, *The Joy of Cryptography*.

# Course Roadmap

- **Weeks 1–5: Private-key encryption.** Perfect secrecy and the one-time pad, pseudorandomness, PRFs, CPA/CCA security, stream and block ciphers (SPNs, Feistel, DES).

(Midterm I: Sep 24)

- **Weeks 6–7: Integrity and hashing.** MACs, authenticated encryption, hash functions, one-way functions.
- **Weeks 8–10: Public-key cryptography.** Groups and hardness assumptions, Diffie–Hellman, El Gamal, RSA, digital signatures and PKI.

(Midterm II: Oct 29)

- **Weeks 11–15: Advanced topics.** Pairings and IBE, commitments, zero-knowledge proofs, succinct proofs, secret sharing, multiparty computation.
- **Final:** Dec 18.

# Questions?

New material!

Please ask questions throughout this class.

# About the Course

- Cryptography is used **everywhere**, with many applications.
- **Course goal**: learn the theoretical basis of the cryptography used in the real world.
  - Learn about key **primitives**.
  - Understand what **security** they provide.
  - Know how to **use** them.
  - Understand “how things work”.
- **Do not**: brew your own crypto!

# About the Course

- Cryptography is used **everywhere**, with many applications.
- **Course goal**: learn the theoretical basis of the cryptography used in the real world.
  - Learn about key **primitives**.
  - Understand what **security** they provide.
  - Know how to **use** them.
  - Understand “how things work”.
- **Do not**: brew your own crypto!

## Crypto mindset

Think carefully about definitions, assumptions, and proofs — not ad hoc tricks.

# Cryptography Historically

- Focused on **private communication** between two parties using previously shared secret information.
- Lack of clarity on what it means to be secure.
- Heuristic, unprincipled design approach.
- Break-and-fix cycle (cat-and-mouse game).



# Modern Cryptography

- More than just confidentiality, e.g. data integrity.
- More tasks: **public-key** cryptography.
- **Rigorous** security definitions.
- Sound mathematical principles for arguing/proving security.
  - E.g. basing it on **factoring**.



# From Art to Science

In summary

Cryptography has developed from an **art** to a **science**.

# Classical Cryptography

- Focused on secret-key / shared-key / private-key / symmetric-key cryptography.
- Start with simple schemes used for centuries.
- Demonstrate the dangers of an unprincipled approach.
- Show why simple tricks are unlikely to work.

# Private-Key Encryption (Confidential Communication)



# Private-Key Encryption (Confidential Communication)

key  
↓  
 $k$

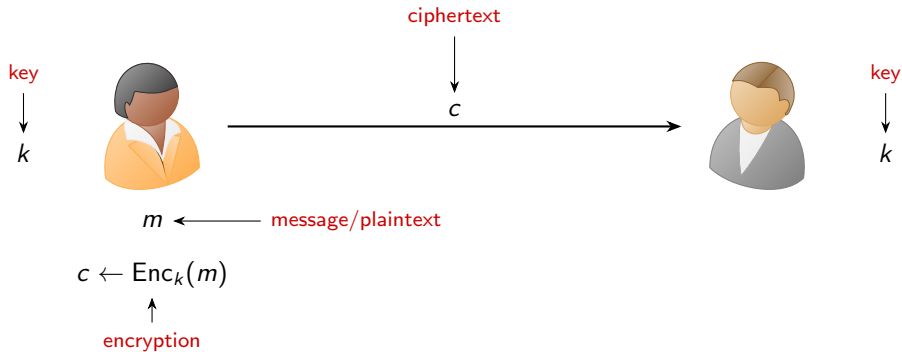


key  
↓  
 $k$

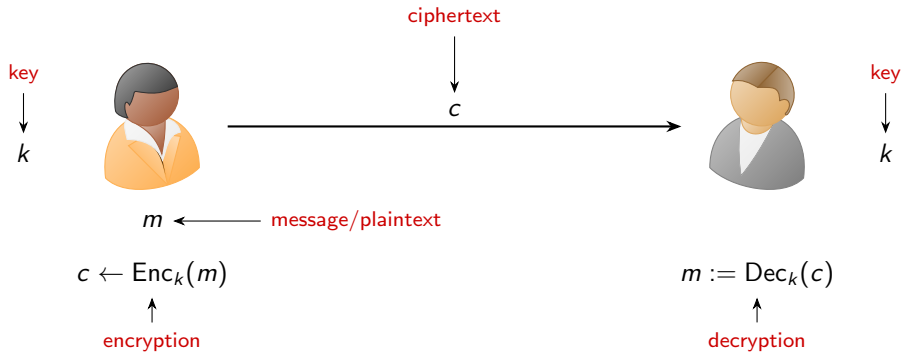
# Private-Key Encryption (Confidential Communication)



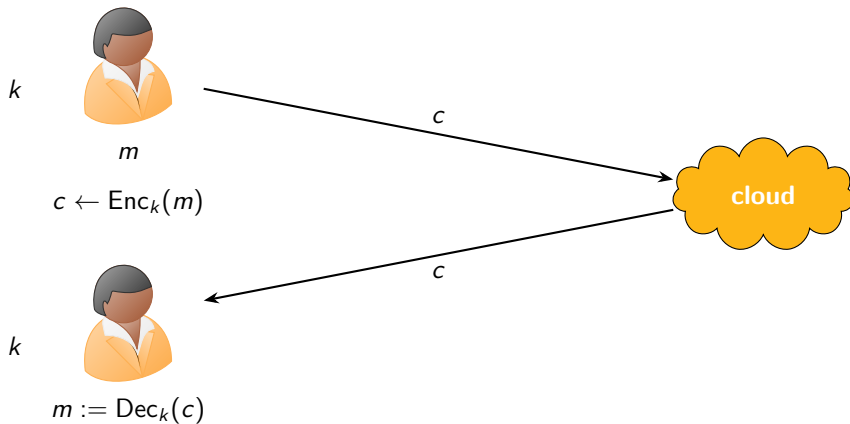
# Private-Key Encryption (Confidential Communication)



# Private-Key Encryption (Confidential Communication)



# Private-Key Encryption (Confidential Self-Communication)



# Private-Key Encryption (Syntax)

## Definition 1 (Private-Key Encryption Scheme)

A **private-key encryption scheme** is a message space  $\mathcal{M}$ , a key space  $\mathcal{K}$ , and algorithms (Gen, Enc, Dec):

- Gen (key generation): outputs  $k \in \mathcal{K}$ .
- Enc (encryption): on key  $k$  and message  $m \in \mathcal{M}$ , outputs ciphertext  $c \leftarrow \text{Enc}_k(m)$ .
- Dec (decryption): on key  $k$  and ciphertext  $c$ , outputs  $m := \text{Dec}_k(c)$  (or “error”).

# Private-Key Encryption (Syntax)

## Definition 1 (Private-Key Encryption Scheme)

A **private-key encryption scheme** is a message space  $\mathcal{M}$ , a key space  $\mathcal{K}$ , and algorithms (Gen, Enc, Dec):

- Gen (key generation): outputs  $k \in \mathcal{K}$ .
- Enc (encryption): on key  $k$  and message  $m \in \mathcal{M}$ , outputs ciphertext  $c \leftarrow \text{Enc}_k(m)$ .
- Dec (decryption): on key  $k$  and ciphertext  $c$ , outputs  $m := \text{Dec}_k(c)$  (or “error”).

## Correctness

For all  $m \in \mathcal{M}$  and all  $k$  output by Gen:  $\text{Dec}_k(\text{Enc}_k(m)) = m$ .

# Private-Key Encryption (Syntax)

## Definition 1 (Private-Key Encryption Scheme)

A **private-key encryption scheme** is a message space  $\mathcal{M}$ , a key space  $\mathcal{K}$ , and algorithms (Gen, Enc, Dec):

- Gen (key generation): outputs  $k \in \mathcal{K}$ .
- Enc (encryption): on key  $k$  and message  $m \in \mathcal{M}$ , outputs ciphertext  $c \leftarrow \text{Enc}_k(m)$ .
- Dec (decryption): on key  $k$  and ciphertext  $c$ , outputs  $m := \text{Dec}_k(c)$  (or “error”).

## Correctness

For all  $m \in \mathcal{M}$  and all  $k$  output by Gen:  $\text{Dec}_k(\text{Enc}_k(m)) = m$ .



$k$  must be kept secret

# Private-Key Encryption (Syntax)

## Definition 1 (Private-Key Encryption Scheme)

A **private-key encryption scheme** is a message space  $\mathcal{M}$ , a key space  $\mathcal{K}$ , and algorithms (Gen, Enc, Dec):

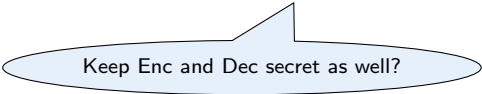
- Gen (key generation): outputs  $k \in \mathcal{K}$ .
- Enc (encryption): on key  $k$  and message  $m \in \mathcal{M}$ , outputs ciphertext  $c \leftarrow \text{Enc}_k(m)$ .
- Dec (decryption): on key  $k$  and ciphertext  $c$ , outputs  $m := \text{Dec}_k(c)$  (or “error”).

## Correctness

For all  $m \in \mathcal{M}$  and all  $k$  output by Gen:  $\text{Dec}_k(\text{Enc}_k(m)) = m$ .



$k$  must be kept secret



Keep Enc and Dec secret as well?

# Kerckhoffs's Principle

## Kerckhoffs's principle

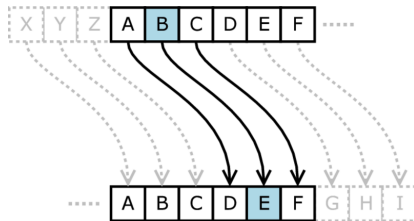
The cipher method must not be required to be secret, and it must be able to fall into the hands of the enemy without inconvenience.

- Only the **key**  $k$  is kept secret.
- Very important in modern cryptography: use open, well-studied tools.



# The Shift Cipher

- Consider encrypting English text.
- Associate 'a' with 0, 'b' with 1, ..., 'z' with 25.
- Key space  $\mathcal{K} = \{0, \dots, 25\}$ .
- To encrypt with key  $k$ , shift every letter by  $k$  positions (with wraparound, mod 26).
- Decryption reverses this.



Example shift by  $k = 3$ .

# The Shift Cipher, Formally

## Definition 2 (Shift Cipher)

- $\mathcal{M} = \{\text{strings over the lowercase English alphabet}\}.$
- $\mathcal{K} = \{0, \dots, 25\}.$
- Gen: choose uniform  $k \in \{0, \dots, 25\}.$
- $\text{Enc}_k(m_1 \cdots m_t)$ : output  $c_1 \cdots c_t$ , where  $c_i := [m_i + k \bmod 26].$
- $\text{Dec}_k(c_1 \cdots c_t)$ : output  $m_1 \cdots m_t$ , where  $m_i := [c_i - k \bmod 26].$

# The Shift Cipher, Formally

## Definition 2 (Shift Cipher)

- $\mathcal{M} = \{\text{strings over the lowercase English alphabet}\}.$
- $\mathcal{K} = \{0, \dots, 25\}.$
- Gen: choose uniform  $k \in \{0, \dots, 25\}.$
- $\text{Enc}_k(m_1 \cdots m_t)$ : output  $c_1 \cdots c_t$ , where  $c_i := [m_i + k \bmod 26].$
- $\text{Dec}_k(c_1 \cdots c_t)$ : output  $m_1 \cdots m_t$ , where  $m_i := [c_i - k \bmod 26].$

## Correctness

For each  $i$ :

$$[c_i - k \bmod 26] = [[m_i + k \bmod 26] - k \bmod 26] = [m_i + k - k \bmod 26] = m_i.$$

## Is the Shift Cipher Secure?

Ciphertext

etarvqogcpuetarvqitcrja

# Is the Shift Cipher Secure?

## Ciphertext

etarvqogcpuetarvqitcrja

- **No** — only 26 possible keys!
  - Try decrypting with every possible key.
  - Only one possibility will “make sense”.
  - Assumes English text has enough structure/redundancy that only one candidate makes sense.
- Example of a **brute-force** (exhaustive-search) attack.

# Summary

- Cryptography has moved from an **art** to a **science**: definitions, assumptions, proofs.
- **Private-key encryption** syntax: (Gen, Enc, Dec) with correctness.
- **Kerckhoffs's principle**: security must rest only on the secret key.
- The **shift cipher**: only 26 keys, so a **brute-force** attack tries them all.

## Reading

Today: KL 1.1–1.3.    Next time: breaking more classical ciphers and their lessons, principles of modern cryptography, and perfect secrecy (KL 1.3–1.4, 2.1).